

Deep Learning - Week 11

1. For which of the following problems are RNNs suitable?

- (a) Generating a description from a given image.
- (b) Forecasting the weather for the next N days based on historical weather data.
- (c) Converting a speech waveform into text.
- (d) Identifying all objects in a given image.

Correct Answer: (a),(b),(c)

Solution: Given an image, generate a description about it: RNNs can be used for this task, but typically as part of a larger architecture. The image would first be processed by a Convolutional Neural Network (CNN), and then an RNN would generate the text description. This is called an encoder-decoder or sequence-to-sequence model.

Given the historical weather data, forecast the weather for the next N days: This is very suitable for RNNs. Weather data is a time series, and RNNs are excellent at processing sequential data and capturing temporal dependencies.

Given a speech waveform, convert it into text: This is also highly suitable for RNNs. Speech recognition involves processing a sequence of audio features and outputting a sequence of characters or words. RNNs (especially when combined with techniques like CTC loss) are very effective for this task.

Given an image, find all objects in the image: This task is primarily suited for Convolutional Neural Networks (CNNs), not RNNs. Object detection in images is typically done using architectures like R-CNN, YOLO, or SSD, which are based on CNNs.

2. Suppose that we need to develop an RNN model for sentiment classification. The input to the model is a sentence composed of five words and the output is the sentiments (positive or negative). Assume that each word is represented as a vector of length 100×1 and the output labels are one-hot encoded. Further, the state vector s_t is initialized with all zeros of size 30×1 . How many parameters (including bias) are there in the network?

Correct Answer: 3992

Solution: **Solution:** To compute the number of parameters in the RNN for sentiment classification, we need to consider the parameters for the following components:

- (a) Input to Hidden State Weights $\mathbf{W}_{xh}$: This weight matrix maps the input word vector to the hidden state.
- (b) Hidden to Hidden State Weights $\mathbf{W}_{hh}$: This weight matrix maps the previous hidden state to the next hidden state.

- (c) Hidden State to Output Weights $\mathbf{W}_{hy}$: This weight matrix maps the hidden state to the output.
- (d) Biases: Bias vectors for both the hidden state and the output.

Given:

- Input word vector size: 100×1
- Hidden state size: 30×1
- Output size: 2×1 (since the sentiment classification is binary: positive or negative)
- Sentence length: 5 words (though it does not affect the parameter count as the same weights are used across all time steps)

Step 1: Compute $\mathbf{W}_{xh}$ (Input to Hidden State Weights)

- The input vector is of size 100×1 , and the hidden state is of size 30×1 .
- Therefore, the weight matrix $\mathbf{W}_{xh}$ has dimensions 30×100 .
- Total parameters in $\mathbf{W}_{xh}$: $30 \times 100 = 3000$

Step 2: Compute $\mathbf{W}_{hh}$ (Hidden to Hidden State Weights)

- The hidden state at time $t - 1$ is of size 30×1 , and the hidden state at time t is also of size 30×1 .
- Therefore, the weight matrix $\mathbf{W}_{hh}$ has dimensions 30×30 .
- Total parameters in $\mathbf{W}_{hh}$: $30 \times 30 = 900$

Step 3: Compute $\mathbf{b}_h$ (Bias for Hidden State)

- The bias for the hidden state is a vector of size 30×1 .
- Total parameters in $\mathbf{b}_h$: 30

Step 4: Compute $\mathbf{W}_{hy}$ (Hidden State to Output Weights)

- The hidden state is of size 30×1 , and the output is of size 2×1 (binary classification).
- Therefore, the weight matrix $\mathbf{W}_{hy}$ has dimensions 2×30 .
- Total parameters in $\mathbf{W}_{hy}$: $2 \times 30 = 60$

Step 5: Compute $\mathbf{b}_y$ (Bias for Output)

- The bias for the output is a vector of size 2×1 .
- Total parameters in $\mathbf{b}_y$: 2

Step 6: Total Parameters Now, let's sum up all the parameters:

$$\text{Total parameters} = \mathbf{W}_{\text{xh}} + \mathbf{W}_{\text{hh}} + \mathbf{b}_{\text{h}} + \mathbf{W}_{\text{hy}} + \mathbf{b}_{\text{y}}$$

$$\text{Total parameters} = 3000 + 900 + 30 + 60 + 2 = 3992$$

Conclusion: The total number of parameters in the RNN is 3992.

3. Select the correct statements about GRUs

- (a) GRUs have fewer parameters compared to LSTMs
- (b) GRUs use a single gate to control both input and forget mechanisms
- (c) GRUs are less effective than LSTMs in handling long-term dependencies
- (d) GRUs are a type of feedforward neural network

Correct Answer: (a),(b)

Solution: a) GRUs have fewer parameters compared to LSTMs This is correct. GRUs have a simpler structure with fewer gates than LSTMs, resulting in fewer parameters.
b) GRUs use a single gate to control both input and forget mechanisms This is correct. GRUs have an update gate that combines the functions of the input and forget gates found in LSTMs.

c) GRUs are less effective than LSTMs in handling long-term dependencies This is generally not correct. GRUs can be as effective as LSTMs in many tasks, including those involving long-term dependencies. Their performance is often comparable to LSTMs.

d) GRUs are a type of feedforward neural network This is incorrect. GRUs are a type of recurrent neural network (RNN), not a feedforward neural network.

4. What is the main advantage of using GRUs over traditional RNNs?

- (a) They are simpler to implement
- (b) They solve the vanishing gradient problem
- (c) They require less computational power
- (d) They can handle non-sequential data

Correct Answer: (b)

Solution: True. GRUs, like LSTMs, help mitigate the vanishing gradient problem by using gating mechanisms (update and reset gates) that regulate the flow of information and gradients over time, allowing them to learn long-term dependencies more effectively than traditional RNNs.

5. The statement that LSTM and GRU solves both the problem of vanishing and exploding gradients in RNN is

- (a) True

(b) False

Correct Answer: (b)

Solution: False. While LSTM (Long Short-Term Memory) and GRU (Gated Recurrent Unit) significantly mitigate the vanishing and exploding gradient problems compared to vanilla RNNs, they do not completely solve these issues. They use gating mechanisms (like the forget, input, and output gates in LSTM, or the reset and update gates in GRU) that help preserve and control the flow of information over long sequences, reducing the impact of vanishing and exploding gradients. However, under certain conditions, especially with very deep networks or extremely long sequences, these problems can still occur.

6. What is the vanishing gradient problem in training RNNs?

- (a) The weights of the network converge to zero during training
- (b) The gradients used for weight updates become too large
- (c) The network becomes overfit to the training data
- (d) The gradients used for weight updates become too small

Correct Answer: (d)

Solution: The vanishing gradient problem is a common issue in training RNNs where the gradients used for weight updates become too small, making it difficult to learn long-term dependencies in the input sequence. This can lead to poor performance and slow convergence during training.

7. What is the role of the forget gate in an LSTM network?

- (a) To determine how much of the current input should be added to the cell state.
- (b) To determine how much of the previous time step's cell state should be retained.
- (c) To determine how much of the current cell state should be output.
- (d) To determine how much of the current input should be output.

Correct Answer: (b)

Solution: The forget gate in an LSTM network determines how much of the previous cell state to forget and how much to keep for the current time step.

8. How does LSTM prevent the problem of vanishing gradients?

- (a) Different activation functions, such as ReLU, are used instead of sigmoid in LSTM.
- (b) Gradients are normalized during backpropagation.
- (c) The learning rate is increased in LSTM.
- (d) Forget gates regulate the flow of gradients during backpropagation.

Correct Answer: (d)

Solution: Due to forget gates controlling the flow, the gradient will only vanish if the previous states didn't contribute during the forward pass. So if the information flows during the forward pass gradient doesn't vanish.

9. We are given an RNN with $\|W\| = 2.5$. The activation function used in the RNN is logistic. What can we say about $\nabla = \left\| \frac{\partial s_{20}}{\partial s_1} \right\|$

- (a) Value of ∇ is very high.
- (b) Value of ∇ is close to 0
- (c) Value of ∇ is 2.5
- (d) Insufficient information to say anything.

Correct Answer: (b)

Solution: The gradient $\nabla = \left\| \frac{\partial s_{20}}{\partial s_1} \right\|$ is influenced by the repeated multiplication of the weight matrix W over multiple time steps. Specifically, it can be approximated as:

$$\nabla \approx \prod_{t=1}^{20} f'(s_t)W$$

where $f'(s)$ is the derivative of the logistic activation function, which is:

$$f'(s) = \sigma(s)(1 - \sigma(s))$$

where $\sigma(s)$ is the sigmoid function. The maximum value of $f'(s)$ occurs at $s = 0$, giving $f'(s) \approx 0.25$.

Now, approximating the gradient magnitude:

$$\begin{aligned}\nabla &\approx (0.25 \times 2.5)^{19} \\ &= (0.625)^{19}\end{aligned}$$

Since $0.625 < 1$, exponentiating it to a large power (like 19) results in a very small value, approaching 0. This suggests that the gradients will diminish significantly, leading to the **vanishing gradient problem**.

Conclusion:

Since ∇ is very small, the correct answer is:

(a) Value of ∇ is close to 0.

10. Select the true statements about BPTT?

- (a) The gradients of Loss with respect to parameters are added across time steps
- (b) The gradients of Loss with respect to parameters are subtracted across time steps
- (c) The gradient may vanish or explode, in general, if timesteps are too large
- (d) The gradient may vanish or explode if timesteps are too small

Correct Answer: (a),(c)

Solution:

Here are the correct statements about Backpropagation Through Time (BPTT):

1. The gradients of Loss with respect to parameters are added across time steps:
- True. In BPTT, gradients are accumulated (added) over all the time steps when calculating the total gradient of the loss with respect to parameters. This is because the same parameters are used at each time step in the recurrent network.
2. The gradients of Loss with respect to parameters are subtracted across time steps:
- False. Gradients are not subtracted; they are added across time steps, as described above.
3. The gradient may vanish or explode, in general, if timesteps are too large: - True. The vanishing or exploding gradient problem is a common issue in RNNs, especially when the number of time steps is large. As the gradient is propagated back through many time steps, it can either shrink exponentially (vanish) or grow exponentially (explode).
4. The gradient may vanish or explode if timesteps are too small: - False. The problem of vanishing or exploding gradients typically arises when the number of time steps is large, not when they are small.